One-Shot Project

Gioele Modica (614923)

---

1. Problem and Algorithm

The computation performs 500 iterations of a sparse matrix-vector product (SpMV) on a square CSR matrix $A \in \mathbb{R}^{N \times N}$. Every **EPOCH_LEN = 25** iterations, the matrix undergoes a circular shift of the rows: logical row **i** reads physical row **src = (i + n − row_shift) % n**. The matrix is never physically modified; only the integer **row_shift**, which increases by a fixed value **shift_rows** at each epoch boundary. Each iteration normalizes the resulting vector (L2 norm), then swaps the input and output buffers. After 500 iterations the Rayleigh quotient **x^T(Ax)** approximates the dominant eigenvalue.

Irregular load
In **-m irregular** mode about **12.5 %** of the rows (the first n/10) contain **40–160×** more non-zero elements than the tail. This causes a very severe load imbalance in static partitions and is the main challenge of parallelization.

Sequential reference (**n = 500 000, nz = 20 000 000, -m irregular**): takes 42.91 s.
Large-scale reference (**n = 2 000 000, nz = 80 000 000**): 217.93 s.

2. Implementation Strategies

2.1 C++ Threads (std::thread + std::barrier)

T worker threads are created only once before the loop and remain active for all 500 iterations. The central synchronization mechanism is a single instance of **std::barrier<Fn>** with a **completion function** that runs serially on each arrival at the barrier:

```
barrier completion (executed once per iteration, a single thread):
  1. epoch update:  row_shift = (row_shift + shift_rows) % n
  2. norm reduction: sum partial_norms[0..T-1]
  3. normalize y:    y[i] *= 1/sqrt(total)    (O(n))
  4. swap(x, y):      pointer swap, O(1)
  5. reset atomic counter for the next iteration
```

This design guarantees **one barrier per iteration**.

Accumulators **thread-local** with padding to 64 bytes to avoid **false sharing** on the array of partial norms.

Static partitioning (NNZ-balanced)
The boundaries are computed only once at startup via binary search on row_ptr, so that each thread owns approximately total_nnz / T non-zeros of the physical CSR rows. The critical problem is that threads iterate over *logical* rows and access the physical rows through **row_shift**. At each epoch **row_shift** changes, rotating the logical→physical mapping and continuously unbalancing the load. After each epoch the “heavy” physical rows (first n/10) are mapped to a different logical range, which

may belong entirely to a single thread. Static balancing is correct only at the initial epoch (**row_shift = 0**).

Dynamic scheduling (atomic work-stealing)
An **std::atomic<size_t> next_chunk** counter is reset to zero at each iteration. Threads call fetch_add(K, relaxed) to claim a chunk of K logical rows.
Any thread that finishes its chunk immediately fetches the next one, eliminating any waiting period. This completely absorbs the epoch-induced imbalance at the cost of a **LOCK XADD** per chunk.

Work-unit granularity (C++ threads, dynamic mode)
The cost of fetching a chunk is a single LOCK XADD instruction, with no heap allocation nor scheduling queues. The resulting overhead is negligible (<1% of total time), making the dynamic version insensitive to K in the range tested $K \in [16, 1024]$. The sharp contrast with OpenMP Tasks confirms that the bottleneck in the latter is the serialized task-creation loop, not the computation granularity. The atomic counter trades fine-grained scheduling control for almost zero overhead per chunk.

2.2 OpenMP Task-Based

The parallel region covers all 500 iterations. Each iteration uses two **#pragma omp single** blocks, each with an implicit barrier:

1. **First single**: optionally updates **row_shift**, then creates $\lceil n/K \rceil$ tasks (one for each chunk of K logical rows). Each task has **firstprivate(cs, ce)** to capture the row range at creation time; without this clause the loop variable would enter a race condition with task execution. The workers immediately start consuming from the task queue while the master is still creating them. **#pragma omp taskwait** inside the single guarantees that all tasks finish before the implicit barrier is crossed.
2. **Second single**: reduces the partial norms, normalizes y, swaps the pointers, resets the accumulators.

Task granularity trade-off
Task creation is serialized in a single thread. With small K the producer cannot keep T consumers busy: at T=16, with K=64 7,812 tasks/iteration are created and at T=32 the producer becomes the bottleneck (30.4 s, worse than T=8). Increasing K reduces the creation overhead: at K=1024 with 488 tasks/iteration the bottleneck is eliminated.

2.3 OpenMP Work-Sharing

Using **#pragma omp for schedule(dynamic, K) reduction(+:norm_sq)** removes the explicit task creation. The reduction clause allocates private copies of **norm_sq** on each thread's stack (intrinsically free of **false sharing**). The **work-sharing** mechanism of omp "**for**" distributes the chunks via a runtime-managed counter, without the single-thread producer bottleneck, ensuring better scaling at **T** large with **K** small.

The epoch update is pre-computed at the end of the iteration inside the single block.

2.4 MPI + OpenMP (Hybrid)

Matrix distribution
Rank 0 generates the complete matrix and distributes it via **MPI_Scatterv** with NNZ-balanced

boundaries (same algorithm as the C++ threads static partitioning). Each rank keeps its own slice of physical rows for all 500 iterations. The offsets of **row_ptr** are normalized to 0 on each rank after the scatter.

Epoch evolution, AD-002 Option A (logical mapping).
The physical row assignments remain fixed. Each rank tracks the **row_shift** shared scalar (same formula as the sequential, no communication). The epoch transition cost is zero; no data movement occurs.

Option B (**physical redistribution with MPI_Alltoallv**)
was discarded because it would require 20 × O(n) redistributions across all nodes, prohibitively expensive at n = 2 M.

Replicated vector x (AD-003)
Each rank keeps the full vector x (n double). Replication costs O(n × P) of memory but completely eliminates communication for accesses to x.

Communication per iteration (two collective operations):

1. **MPI_Allreduce(local_sq → global_sq, MPI_SUM)**: O(1) data, O(log P) latency.
   This operation computes the global L2 norm; each rank normalizes its own local slice of y independently.
2. **MPI_Allgatherv(y_local → phys_buf) + cyclic_copy:** O(n) data per iteration.
   Each rank contributes its own normalized slice of y; **cyclic_copy** reassembles **x** accounting for **row_shift** via two **memcpy** calls instead of n modular divisions.

SpMV intra-rank uses **#pragma omp parallel for schedule(dynamic, K) reduction(+:local_sq)** with **T** threads per **rank**, handling the load imbalance within the rank on the irregular input. MPI calls are made only by the master thread (**MPI_THREAD_FUNNELED**).

Optimization:
The **MPI** implementation works entirely in physical space during the SpMV: each rank computes **y_local[j] = A_local[j,:] · x** for its own physical rows without ever performing the operation **(i + n - row_shift) % n** in the inner loop. The logical mapping is applied only once, in a vectorized way, via **cyclic_copy** at the time of reassembling **x** (two contiguous memcpy). This eliminates one modular division per row in the most expensive section of the code, improving throughput on large matrices (n=2M → 2M modular divisions avoided per iteration).

3. Correctness Verification

All implementations were verified against the sequential reference with two criteria:

1. **Relative difference of the Rayleigh value** < 1×10^{-10}
2. **Element-by-element diff** (one-liner awk, threshold 1×10^{-10}, empty output = PASS):

Measured Rayleigh values (n = 500 000, nz = 20 000 000, -m irregular, seed = 111):

Implementation	rayleigh	|Δ|/|ref|
Sequential	4.223260535478325	—
Dynamic threads T=4	4.223260535478293	7.6×10^{-15}
Dynamic threads T=16	4.223260535478219	2.5×10^{-14}
OMP Tasks K=1024 T=16	4.223260535478210	2.7×10^{-14}
OMP Work-Sharing T=16	4.223260535478214	2.6×10^{-14}

MPI+OpenMP (n = 2 000 000, nz = 80 000 000, -m irregular):

Configuration	rayleigh	|Δ|/|ref|
Sequential (reference)	4.223544292121269	—
MPI np=4 T=4 (1 node)	4.223544292121007	6.2×10^{-14}
MPI np=8 T=4 (2 nodes)	4.223544292121087	4.3×10^{-14}

All deviations are in the range 10^{-14}–10^{-13}, four orders of magnitude below the 1×10^{-10} threshold. These differences are explained by the non-associativity of floating-point arithmetic: parallel reductions accumulate partial sums in a different order than the sequential reference. The checksum field (**XOR hash of x**) may differ for the same reason and is not a correctness criterion.

4. Performance Single-Node

Configuration: spmcluster node, n = 500 000, nz = 20 000 000, -m irregular.
Baseline: sequential = 42.91 s. All times are medians over 3 runs.

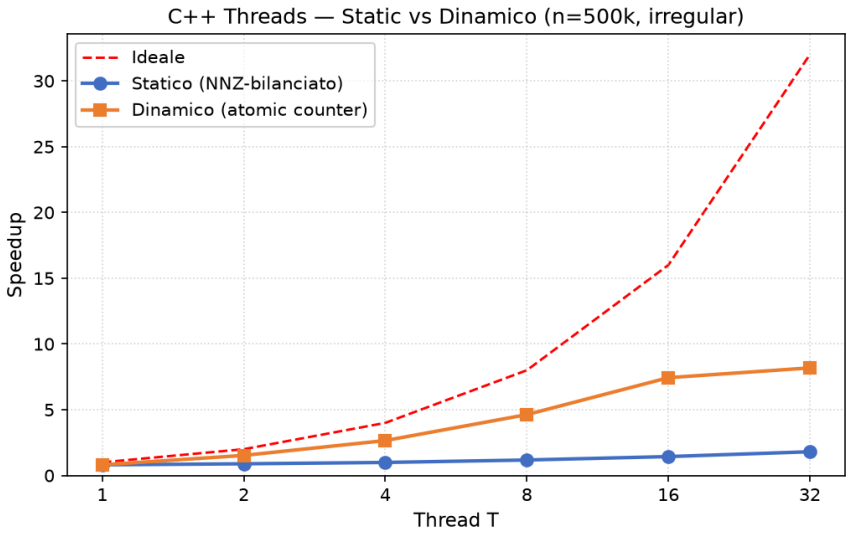
4.1 Static vs Dynamic Partitioning (C++ Threads)

T	Static (s)	Speedup	Dynamic (s)	Speedup	Efficiency (dyn)
1	52.97	0.81	53.14	0.81	0.807
2	48.27	0.89	28.10	1.53	0.763
4	43.27	0.99	16.16	2.66	0.664
8	36.45	1.18	9.27	4.63	0.578
16	29.85	1.44	5.77	7.44	0.465
32	23.78	1.80	5.25	8.18	0.256

Static partitioning fails despite NNZ balancing
The partition is computed once during initialization on the physical CSR rows. Each thread iterates over a fixed range of *logical* rows, accessing the physical rows through the current row_shift. At each epoch transition the logical → physical mapping rotates: the logical range that accessed the light tail (low NNZ) now accesses the heavy head and vice versa. Over the 20 epoch transitions the work per

thread is sampled uniformly across all weight zones, but in any given epoch a single thread holds almost all the heavy rows, causing an O(T) imbalance. The average speedup of 1.44× at T=16 reflects this epoch-averaged imbalance.

Dynamic scheduling completely mitigates the imbalance
Since each thread immediately fetches the available chunk when it finishes its own, the epoch-induced hot spots are distributed across the thread pool within a single iteration. The 7.4× speedup at T=16 is close to ideal on 16 physical cores, confirming that computation dominates once the scheduling overhead is removed. Efficiency drops at T=32 (HT) because two logical threads share the same execution unit.



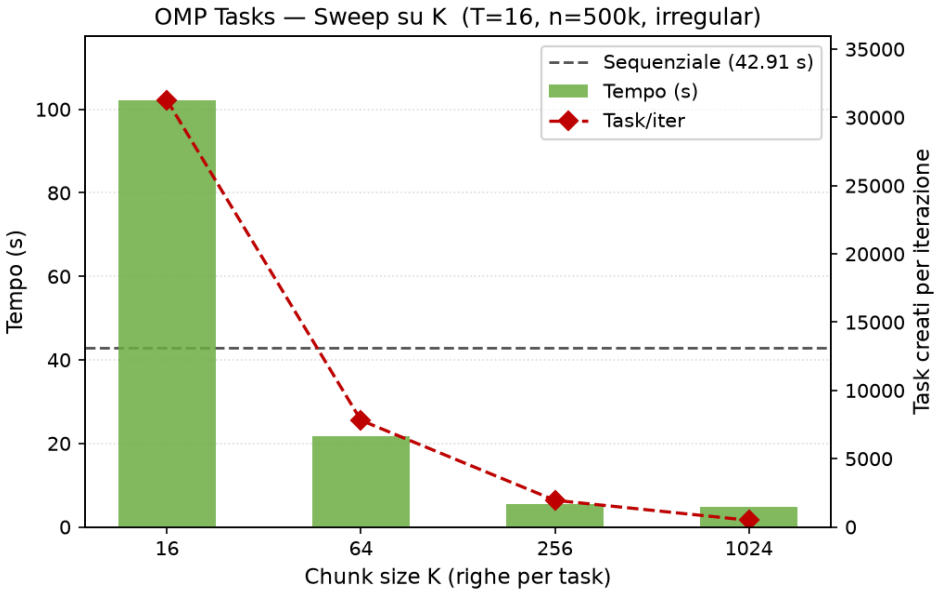
4.2 OMP Task Granularity (sweep over K, T=16 fixed)

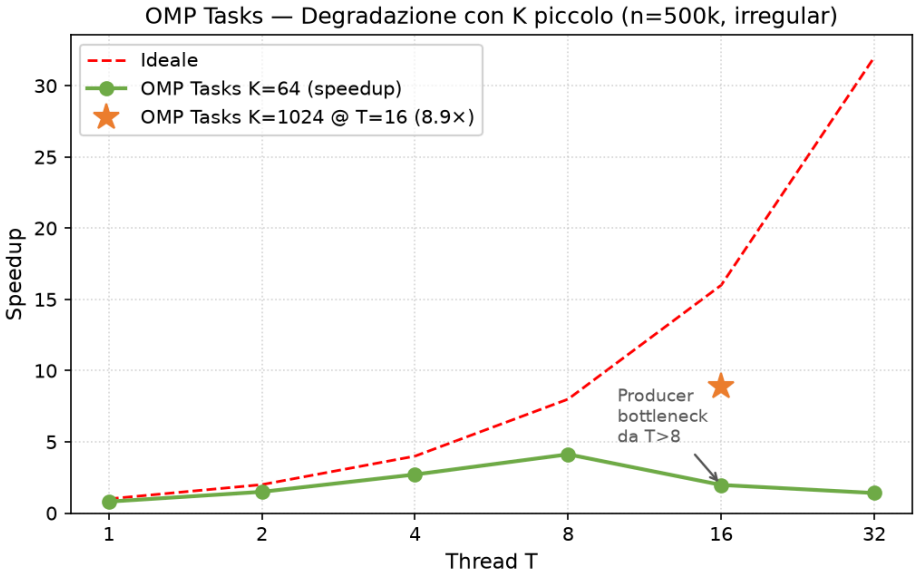
K	Task/iter	Time (s)	Notes
16	31,250	102.15	Serialized task creation → **worse than sequential**
64	7,812	21.70	Still overhead-dominated
256	1,953	5.52	Computation begins to dominate execution time.
1024	488	4.83	**Best**

Analysis
Task creation is serialized in the omp single block.
With K=16 ~31k tasks/iteration are generated:
the scheduling overhead (~100–500 ns/task) dominates over the computation, leading to times worse than sequential. At K=1024 the tasks are only ~488/iteration, each with a computation volume

far exceeding the creation overhead (~200 ns). The inflection point sits around K=256, beyond which computation dominates again.

The same degradation appears in the sweep over T at K=64 fixed: T=8 gives 10.4 s, T=16 regresses to 21.7 s, T=32 to 30.5 s. With 16 consumers, the single-thread producer creating 7,812 tasks/iteration becomes the bottleneck.

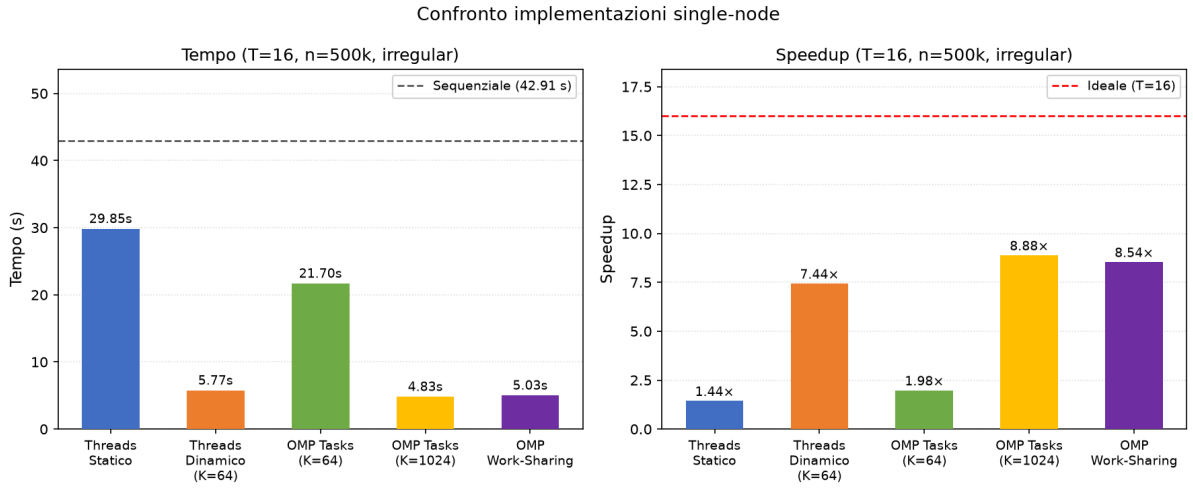




4.3 Implementation Comparison at T=16 (n=500k, irregular)

Implementation	Time (s)	Speedup	Notes
Static Threads	29.85	1.44×	NNZ balancing broken by epochs
Dynamic Threads (K=64)	5.77	7.44×	Atomic fetch_add overhead
OMP Tasks (K=64)	21.70	1.98×	Task creation bottleneck
OMP Tasks (K=1024)	4.83	**8.88×**	K best
OMP Work-Sharing (K=64)	5.03	**8.54×**	No creation bottleneck

OpenMP Tasks with the optimal chunk size slightly outperform OpenMP Work-Sharing.
This may be due to the fact that the task runtime can perform fine-grained work-stealing from the thread queue, smoothing out the micro-irregularities within a chunk.
OMP Work-Sharing provides nearly identical results with less need for tuning: it completely avoids sensitivity to K (no single-thread producer) and achieves good balancing via the runtime's shared-counter mechanism.



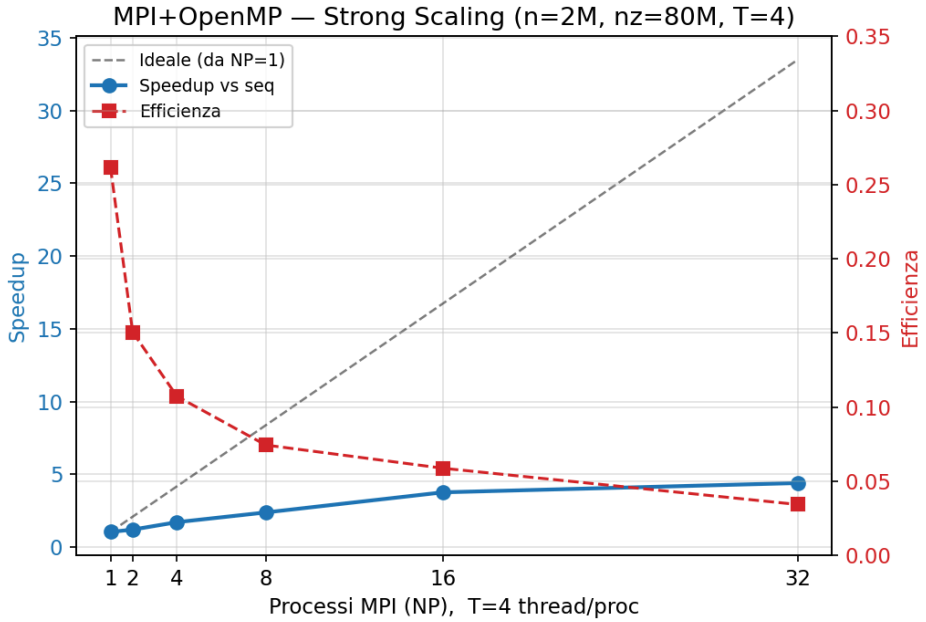
5. Performance MPI+OpenMP

Configuration: n = 2 000 000, nz = 80 000 000, -m irregular.
Sequential reference = **217.93 s**. T = 4 threads per process for all strong/weak scaling.

5.1 Strong Scaling (fixed problem size)

NP	Nodes	Total cores	Time (s)	Speedup vs seq	Efficiency
1	1	4	208.35	1.05×	0.261
2	1	8	180.93	1.20×	0.151
4	1	16	126.95	1.72×	0.107
8	2	32	91.43	2.38×	0.074
16	4	64	57.88	3.76×	0.059
32	8	128	49.49	4.40×	0.034

Strong scaling efficiency is low: 4.4× on 128 logical cores.
The main cause is MPI_Allgatherv, which transfers O(n) at each iteration regardless of the number of processes.
This is a *constant communication volume*, that does not decrease as NP increases, so adding processes does not reduce the communication cost.



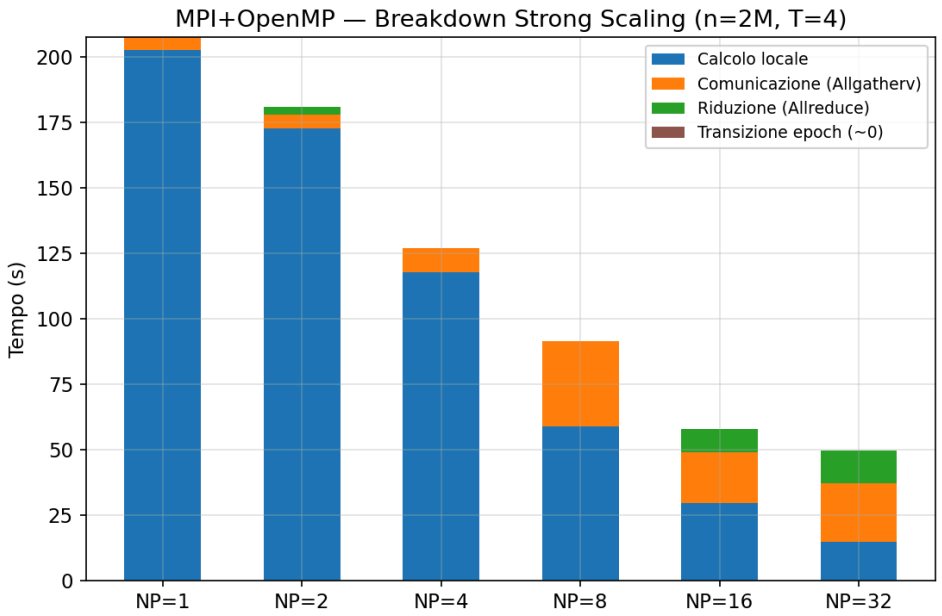
5.2 Execution Time Breakdown (Strong Scaling)

NP	Local compute (s)	Comm/Allgatherv (s)	Allreduce (s)	Epoch (s)	Total (s)
1	202.6	4.89	0.002	~0	208.4
4	117.6	9.19	0.13	~0	127.0
8	58.9	32.4	0.07	~0	91.4
16	29.6	19.2	9.1	~0	57.9
32	14.7	22.4	12.4	~0	49.5

Three phases emerge as NP grows:

1. **NP ≤ 4 (intra-node)**: local compute dominates. Communication (Allgatherv in shared memory) adds ~9 s of overhead. Allreduce latency is negligible (<0.1 s). The epoch transition cost is zero, the logical mapping avoids any data movement.
2. **NP = 8 (first inter-node jump)**: Allgatherv jumps to 32.4 s, with the effective inter-node bandwidth (~250 MB/s) about 3.5× lower than the intra-node one. Local compute halves to 58.9 s. Communication now represents **35 %** of the total time.
3. **NP ≥ 16 (multi-node)**: both Allgatherv and Allreduce grow. At NP=32 local compute is only 14.7 s (29 % of total), while communication (22.4 s) and reduction (12.4 s) together represent **71 %** of the execution time. Adding processes simply shifts time from computation to communication, diminishing returns.

Epoch transition cost = 0 in all configurations, confirming that Option A (logical mapping) is the correct choice. Option B (MPI_Alltoallv redistribution at each epoch) would add 20 × O(n) collective operations, estimated at ~100–200 s of additional overhead based on the measured Allgatherv bandwidth.



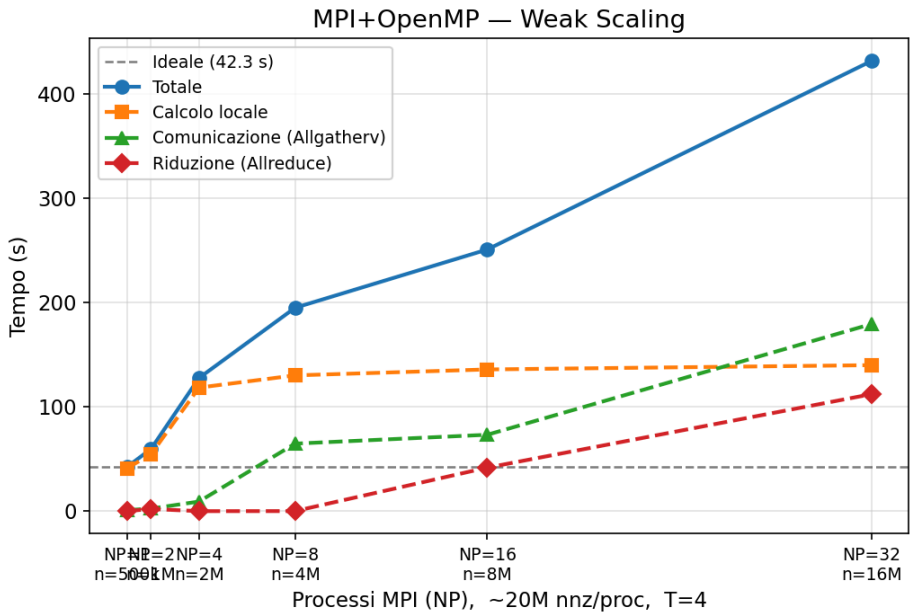
5.3 Weak Scaling (constant work per process: ~20M nnz)

NP	Nodes	n	Time (s)	Compute (s)	Comm (s)	Reduction (s)
1	1	500k	42.3	41.1	0.97	0.002
2	1	1M	59.6	54.5	2.44	~2.1
4	1	2M	127.6	118.4	9.21	0.009
8	2	4M	194.9	130.1	64.7	0.065
16	4	8M	250.7	135.8	73.2	41.7
32	8	16M	431.4	139.9	179.3	112.1

Weak scaling deviates significantly from the ideal behavior: total time grows from 42.3 s (NP=1) to 431.4 s (NP=32), a slowdown of about 10× against a 32× increase in resources.

The reason is that MPI_Allgatherv sends n_total doubles at each iteration, and n_total = n_per_proc × NP grows linearly with NP. This is a fundamental architectural limit of the replicated-x design: in weak scaling, the Allgatherv data volume scales linearly with NP. At NP=32, the total volume (~64 GB) completely saturates the inter-node bandwidth, explaining the 179 s of measured communication.

Local compute grows only slightly (41.1 → 139.9 s) due to the increase in n that causes cache pressure and NUMA effects, confirming that computation is not the bottleneck.



5.4 NP × T Interaction (n=2M, nz=80M)

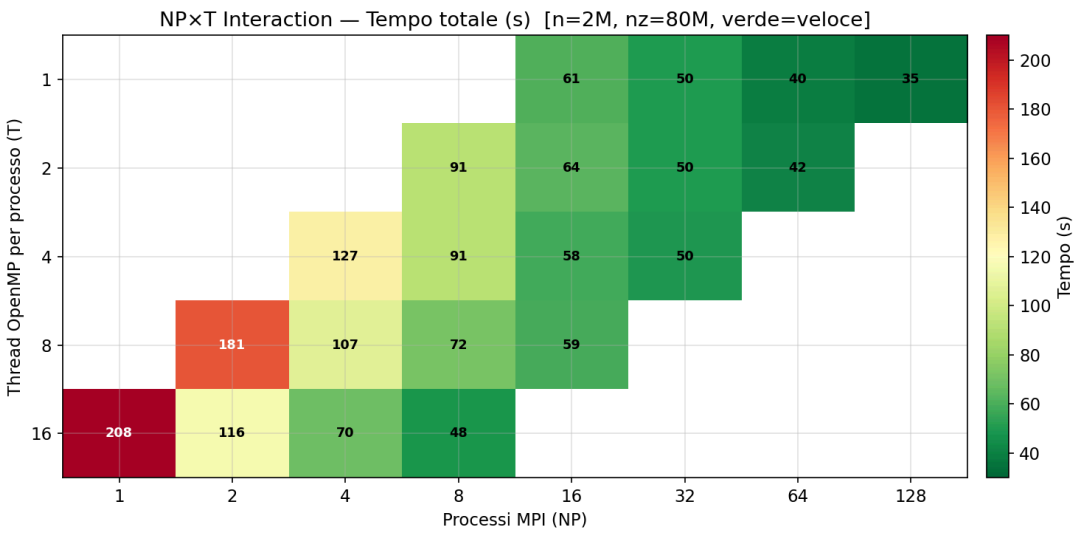
On a fixed problem, different (NP, T) combinations with the same total number of logical cores produce different performance because they affect the compute/communication ratio:

Total cores	NP	T/proc	Nodes	Time (s)	Compute (s)	Comm+Red (s)
16	1	16	1	208.1	202.4	4.89
16	2	8	1	180.9	171.5	9.25
16	4	4	1	127.1	117.6	9.35
16	8	2	1	90.6	79.6	10.96
16	16	1	1	61.2	41.1	20.1
32	2	16	2	116.1	101.1	15.3
32	4	8	2	107.0	85.7	21.4
32	8	4	2	91.3	58.7	32.5
32	16	2	2	64.4	39.3	25.0

Total cores	NP	T/proc	Nodes	Time (s)	Compute (s)	Comm+Red (s)
32	32	1	2	50.1	20.2	29.9
64	4	16	4	70.0	50.6	18.9
64	8	8	4	71.6	42.7	29.0
64	16	4	4	57.9	29.6	28.3
64	32	2	4	50.0	20.0	29.6
64	64	1	4	39.5	10.2	29.5
128	8	16	8	48.2	25.9	22.2
128	16	8	8	59.4	21.9	37.4
128	32	4	8	49.5	14.7	34.8
128	64	2	8	41.8	10.1	31.6
128	128	1	8	34.8	5.1	29.7

Main observations:

1. **More processes, fewer threads per process is generally better** for the same total cores. At 16 total cores, np=16 t=1 (61.2 s) beats np=1 t=16 (208.1 s). The reason: more MPI processes = smaller n slice per process = faster local OpenMP computation. The Allgatherv volume (O(n_total)) is identical in both cases.
2. **The compute fraction vanishes at high NP.** At NP=128, T=1 (8 nodes): local compute = 5.1 s vs total = 34.8 s. Only **14.7 %** of the time is spent in computation; the remaining 85.3 % is communication. Adding further resources would provide no additional benefit since computation is already negligible.
3. **At fixed nodes (e.g., 8 nodes), the optimal configuration is NP=128 T=1** (34.8 s) instead of NP=8 T=16 (48.2 s). This is counterintuitive: more MPI processes with single-threaded ranks outperform fewer processes with multi-threaded ranks. The explanation: intra-node MPI communication (shared memory) is very fast, so np=16 per node × 8 nodes = 128 processes at T=1 adds ~29.7 s of Allgatherv overhead (same data volume as the T=16 case) but reduces computation from 25.9 s to 5.1 s.
4. **MPI_Allreduce becomes significant at high NP** (12.4 s at NP=32, 14.6 s at NP=64, 10.9 s at NP=128). This O(log P) latency cost grows because the reduction latency across many nodes is not negligible when 500 reductions accumulate.



6. Bottleneck Analysis and Scalability Limits

Primary bottleneck: MPI_Allgatherv, O(n) per iteration

Each iteration requires reconstructing x on all ranks by gathering the normalized slices of y_local. The total data volume is n doubles regardless of NP. With n=2M, the total data transferred per run is the dominant cost at NP≥8. This is an intrinsic limit of the replicated-x design, not a tuning problem.

The alternative (halo exchange, each rank fetches only the columns of x needed for its rows) would reduce communication for structured sparsity patterns, but for irregular random column indices the working set of x entries per row is not correlated with the rank boundaries, requiring a non-uniform all-to-all communication that could be even slower in practice.

Secondary bottleneck: MPI_Allreduce, O(log P) latency per iteration

The 500 Allreduce operations cost about 25 ms each at NP=32 (12.4 s total): latency is dominated by the depth of the communication tree rather than by bandwidth, and represents up to 25% of the total time at NP=32.

Imbalance bottleneck (single-node): epoch rotation

NNZ-balanced static partitioning fails on irregular input because the logical→physical mapping rotates at each epoch, redistributing the heavy rows among the threads. Dynamic scheduling (atomic counter) eliminates this problem with negligible overhead (below 4% of the computation at T=16).

Memory bandwidth saturation

At n=2M, local compute scales less than expected because the 80M NNZ of double values (~960 MB) do not fit in any cache level: the SpMV is completely bandwidth-bound. Hyperthreading (T=32) provides a marginal improvement over T=16, consistent with bandwidth sharing among logical cores.

7. Conclusions

Metric	Result
Best single-node (T=16, n=500k)	OMP Tasks K=1024: **8.88×** speedup
Strong scaling (NP=32, T=4, n=2M)	**4.40×** speedup on 128 cores
Weak scaling (NP=32, T=4)	**10× slowdown** (communication-dominated)
Epoch transition overhead	**~0** (logical mapping, no redistribution)

The single-node implementations demonstrate that dynamic work distribution is essential for irregular SpMV: NNZ-balanced static partitioning is broken by epoch rotation, while dynamic chunking achieves nearly linear scaling up to the number of physical cores. OMP Work-Sharing offers the best balance between performance and robustness (no K tuning required); OMP Tasks at the optimal chunk size (K = 1024) is slightly better but sensitive to the granularity parameter.

The distributed MPI+OpenMP implementation is fundamentally limited by the O(n) per-iteration Allgatherv. Although the logical mapping strategy completely eliminates the redistribution cost, the replicated-x scheme requires transmitting n doubles at each iteration. At n=2M and 8 nodes, communication consumes 71 % of the execution time. The practical scalability limit for this algorithm with replicated x is about NP=8–NP=16 (4× speedup), beyond which each additional process reduces computation time by less than the communication overhead it introduces.

A scalable implementation would require reducing the per-iteration communication from O(n) to O(n/P) per rank, which implies partitioning x or adopting a gather-scatter approach that exchanges only the x entries actually needed by each rank, trading communication volume for communication complexity.